KABC Benchmark Results: Experiment 1 (Gaussian Mixtures), Experiment 2 (Identical Means, Different Shapes), and Experiment 3 (MovieLens-1M)

Kernel-based Active Clustering of Bandit Arms
Thuot, Vogt, Ghoshdastidar, Verzelen (arXiv:2601.07535)

September 14, 2026

This report summarizes the experimental setup, baseline algorithms, results, and conclusions for Experiments 1, 2, and 3 of the `kernel-active-bandit` benchmark suite. Full design rationale lives in `docs/experiments.md`; this document summarizes what was actually run and what the real (non-smoke-test) results show.

Run provenance. `results/exp1/results.parquet` holds 10,400 rows (13 grid points × 2 budget multipliers × 4 algorithms × 100 reps, ≈24.4 hours wall-clock). `results/exp2/results.parquet` holds 10,500 rows (21 grid points across three instance families, ≈19 hours wall-clock). `results/exp3/results.par` holds 550 rows (2 grid points, 100 reps per algorithm except TaS-FW's 50, ≈66 minutes wall-clock). All three were generated by `scripts/run_experiment.py` using calibrated threshold constants (Section 3). Figures were regenerated from these files by `notebooks/01_gaussian_mixtures.ipynb`, `notebooks/02_same_mean_shapes.ipynb`, and `notebooks/03_movielens.ipynb`.

Contents

1 Experimental Setup **2**
1.1 Experiment 1 — Gaussian mixtures: competitiveness in the parametric regime . . . 2
1.2 Experiment 2 — Identical means, different shapes: nonparametric separation 3
1.3 Experiment 3 — MovieLens: real rating distributions 4
1.4 Experiment 4 — MovieLens random pairs: robustness across the real-data pair distribution 4

2 Baseline Algorithms **6**
2.1 KABC and KABC-SG (this paper's own methods) 6
2.2 ACB — Active Clustering with Bandit feedback 6
2.3 TaS-FW — Track-and-Stop with Frank-Wolfe 6
2.4 Unif-KKM — uniform sampling + kernel k-means (folklore baseline) 6

3 Calibration **7**

4 Results — Experiment 1 **7**
4.1 Guaranteed budget (budget_multiplier = 1.0): does the theory hold? 7
4.2 Restricted budget (budget_multiplier = 0.25): graceful degradation? 7

5 Results — Experiment 2 **11**
5.1 `moment_matched_mixture` (2b) 11
5.2 `scale_pair` (2a) 12
5.3 `ring_blob` (2c) 13
5.4 Price of adaptivity 14

6 Results — Experiment 3 **14**

7 Results — Experiment 4 **16**

8 Inferences and Conclusions **18**

1 Experimental Setup

1.1 Experiment 1 — Gaussian mixtures: competitiveness in the parametric regime

Claim tested. In the mean-separated Gaussian setting (home turf of mean-based methods like ACB), KABC's budget is within a modest factor of ACB's; budget scales as $s_*^{-2}\log(N/\delta)$ (the paper's Theorem 3.1).

Instance family. `GaussianMixtureEnv`: N arms in $K = 3$ balanced clusters, arm i in cluster k distributed as $\mathcal{N}(\mu_k, \sigma^2 I_d)$, $\sigma = 1$. Cluster means $\mu_k = \frac{\Delta}{\sqrt{2}}(e_{2k-1} + e_{2k})$, giving each pair of cluster means an actual Euclidean separation of $\Delta\sqrt{2}$.

Grid. One knob swept at a time, others held at the base config N=60, K=3, d=10, Δ=3:

- $\Delta \in \{1, 1.5, 2, 3, 4, 6\}$ (6 points — the headline sweep)
- $d \in \{6, 10, 50, 100\}$ (4 points; $d = 2$ excluded, construction requires $d \geq 2K = 6$)
- $N \in \{30, 60, 120\}$ (3 points)

13 distinct grid points total (the base config is each sweep's own midpoint, so it is run identically under all three sweep tags — a documented, accepted redundancy).

Budget regimes. Every grid point above is run under two multipliers:

- budget_multiplier = 1.0: the δ-PAC *guaranteed budget*, budget_cap = $200\,N\,s_*^{-2}\log(N/\delta)$, s_* computed numerically per grid point.
- budget_multiplier = 0.25: a new *restricted budget* pass, same grid, budget cut to a quarter. Motivation: real deployments of active clustering often cannot afford the full guaranteed-budget amount; this measures graceful degradation under a hard 4× cut, complementary to (not a replacement for) the guaranteed-budget story.

This gives $13 \times 2 = 26$ total grid points.

Protocol. $\delta = 0.05$ throughout. 100 Monte Carlo repetitions per (grid point, algorithm). seed = $1000 \cdot$ exp_id + rep. Pilot pulls ($n_{\text{pilot}} = 5$/arm, median-heuristic bandwidth estimation) count toward the reported τ.

Algorithms. `kabc`, `kabc-sg`, `acb`, `unif-kkm` (Section 2). `tas-fw` is *not* run here — its finite-alphabet assumption does not hold for continuous Gaussian instances.

1.2 Experiment 2 — Identical means, different shapes: nonparametric separation

Claim tested. Mean-based active clustering methods (ACB) cannot separate clusters that share a mean but differ in higher-order distributional structure (variance, shape, or geometry); KABC's kernel/MMD machinery can. This is the synthetic complement to Experiment 3's real-data version of the same claim.

Three instance families, $N = 40$, $K = 2$ throughout:

- `scale_pair` (2a): cluster A $= \mathcal{N}(0, 1)$, cluster B $= \mathcal{N}(0, \sigma_B^2)$ — same mean, different *variance*. Swept: $\sigma_B \in \{1.5, 2, 3, 4\}$.
- `moment_matched_mixture` (2b): cluster A $= \mathcal{N}(0, 1)$, cluster B $= 0.5\,\mathcal{N}(-a, b^2) + 0.5\,\mathcal{N}(a, b^2)$, $b^2 = 1 - a^2$ — same mean *and* variance, different *shape* (unimodal $\to$ bimodal as $a \to 1$). Swept: a, with an **asymmetric range by budget regime** (see below).
- `ring_blob` (2c): cluster A = a 2D ring (radius 2, Gaussian noise $\sigma = 0.1$), cluster B $= \mathcal{N}(0, I_2)$, both centered at the origin — same mean, different *2D geometry*. No sweep (one configuration).

Budget regimes and the `moment_matched_mixture` asymmetry. As in Experiment 1, every family is run at both budget_multiplier = 1.0 (guaranteed budget, budget_cap $= 200\,N\,s_*^{-2}\log(N/\delta)$) and 0.25 (restricted budget). For `moment_matched_mixture` specifically, the two regimes cover *different* a-ranges: guaranteed budget covers only $a \in \{0.8, 0.85, 0.9, 0.95\}$, while restricted budget covers the full $a \in \{0.5, 0.6, 0.7, 0.8, 0.85, 0.9, 0.95\}$. This is deliberate, not missing data: direct diagnostic runs (this session, not part of the main grid) showed `kabc` hits an engineering safeguard — a Gram-matrix memory guard added to `kabc.py`'s doubling-trick loop to bound worst-case memory use — on most seeds for $a \in \{0.5, 0.7\}$ at *either* budget regime (the guard trips on Gram-matrix size at a given iteration, not on remaining budget, so shrinking the budget does not avoid it). `kabc` never reaches a real δ-PAC stopping decision at these points, so a τ/error comparison there would only be reporting safeguard trips, not convergence behavior — excluded from the guaranteed-budget pass for that reason. They remain in the restricted-budget pass because ARI on the best partition-so-far is still well-defined even when `stopped=False` (§3's sibling generalization in `docs/experiments.md` §0.4), giving the only view in this report of the full a-range's degradation.

Protocol. $\delta = 0.05$. 100 reps per (family, grid point, algorithm). `kabc-oracle` (needs the true s_*^2 as an oracle input) is included only where `budget_cap` is s_*-formula-derived, i.e. every Experiment 2 grid point.

Algorithms. `kabc`, `kabc-sg`, `kabc-oracle`, `acb`, `unif-kkm`. `acb` is expected to be mean-blind and never stop on every family here (all three are same-mean by construction), by design, not as a failure.

Calibration reuse. As elsewhere, `kabc`/`kabc-sg`/`acb` reuse the single frozen calibration from Section 3 (calibrated against Experiment 1's `gaussian_mixture` family) rather than being re-calibrated per family. Section 5 shows this matters: `kabc-sg`'s calibrated constant, in particular, generalizes poorly to all three Experiment 2 families.

1.3 Experiment 3 — MovieLens: real rating distributions

Claim tested. The "same mean, different shape" phenomenon that mean-based methods cannot see (demonstrated on synthetic data in Experiment 2) also occurs in real data (polarizing vs. consensus movies), and KABC exploits it where mean-based methods fail.

Data. MovieLens-1M (`data/ml-1m/`), filtered to movies with ≥ 1000 ratings.

Prototype selection (deterministic, scripted — `MovieLensEnv`):

1. For each retained movie, compute its empirical rating distribution on $\{1, \ldots, 5\}$ and pairwise MMD^2 (RBF kernel, median-heuristic bandwidth).
2. Find the pair maximizing MMD^2 subject to $|\text{mean gap}| \leq 0.05$ — the "polarizing vs. consensus" same-mean pair.
3. Add $K - 2$ further prototypes greedily maximizing minimum pairwise MMD^2.
4. $K = 4$ prototypes total, 10 arms/prototype ($N = 40$, main instance). Pulling an arm draws one rating with replacement from its prototype's pool.

On the real data, the same-mean pair selected was *Clear and Present Danger* (1994, mean rating 3.73) vs. *South Park: Bigger, Longer and Uncut* (1999, mean rating 3.76) — a 3-star-consensus family drama against a bimodal love-it-or-hate-it comedy: essentially the same mean rating, very different distribution shape (Figure 13, left panel).

A separate, much smaller *reduced* instance ($K = 3$ prototypes, 2 arms each, $N = 6$) is built so TaS-FW's finite-alphabet assumption holds natively (MovieLens ratings *are* a finite alphabet, $\{1, \ldots, 5\}$), letting it run as a fair baseline.

Budget. Main instance: fixed budget_cap = 2,000,000 pulls (no s_*-based formula — s_* is not in closed form for real, empirical rating distributions). Reduced instance: budget_cap = 20,000.

Protocol. $\delta = 0.05$. 100 reps/algorithm (50 for TaS-FW). No budget-multiplier sweep for Experiment 3.

Algorithms. Main instance — `kabc`, `kabc-sg`, `acb`, `unif-kkm`. Reduced instance — `tas-fw`, `kabc`.

1.4 Experiment 4 — MovieLens random pairs: robustness across the real-data pair distribution

Claim tested. Experiment 3 uses one fixed, cherry-picked $K = 4$ MovieLens instance. Experiment 4 asks whether KABC's advantage (and ACB's/TaS-FW's relative behavior) is an artifact of that one instance, or holds up when the movie pair is randomly resampled every repetition across

the real diversity of the MovieLens catalog, under three pairing regimes that vary how much mean separation is present and whether the second movie is cherry-picked (max MMD^2) or sampled uniformly.

Data. `data/ml-1m-207/` — the 207 MovieLens-1M movies with ≥ 1000 ratings, pre-filtered so per-rep pair sampling does not re-load/re-filter the full ratings file every repetition. Same discrete rating support $\{1, \ldots, 5\}$ and RBF-on-ratings kernel as Experiment 3, with the same fixed median-heuristic bandwidth (bandwidth = 1.0) computed once over the full 207-movie pool.

Instance. $N = 8$ arms, $K = 2$ clusters, 4 arms per prototype (movie). For each repetition, movie A is drawn uniformly at random from the 207 movies; movie B is chosen from A according to one of three version-specific rules below. `MovieLensPairEnv` then resamples with replacement from each movie's real rating pool to build that repetition's arms. If no eligible B exists for a drawn A, a fresh A is redrawn deterministically until a valid pair is found.

Version-specific pairing rules.

`exp4_1` — **tight mean gap, MMD-maximizing.** Candidates: all movies with $|\text{mean}_A - \text{mean}_B| \leq 0.1$. $B = \arg\max \text{MMD}^2$ among candidates — the same "polarizing vs. consensus, cherry-picked for maximum shape difference at matched mean" construction as Experiment 3, re-drawn fresh every repetition.

`exp4_2` — **moderate mean gap, uniform.** Candidates: all movies with $0.1 \leq |\text{mean}_A - \text{mean}_B| \leq 0.5$. B sampled uniformly at random among candidates.

`exp4_3` — **tight-ish mean gap, uniform.** Candidates: all movies with $|\text{mean}_A - \text{mean}_B| \leq 0.2$. B sampled uniformly at random among candidates — an ablation of `exp4_1` that keeps a similarly tight mean gap but removes the MMD-maximizing cherry-pick.

Budget. Fixed budget_cap = 20,000 pulls for every algorithm (not the s_*-formula type used elsewhere — not meaningful at this instance scale). $\delta = 0.05$, matching every other experiment. Calibrated c_{cal} constants (`kabc`, `kabc-sg`, `acb`) are reused from the existing frozen calibration, per the "calibrate once, reuse everywhere" protocol — Experiment 4's instance family (random MovieLens pairs) is distributionally different from what those constants were calibrated against, so any resulting miscalibration is reported as a finding, not silently re-tuned.

Algorithms. `kabc`, `kabc-sg`, `acb`, `tas-fw`, and two non-adaptive uniform-sampling baselines, `unif-kkm-1x`/`unif-kkm-2x`, whose `budget_cap` is set *per repetition* to $\min(\tau_{\texttt{kabc}}, 20{,}000)$ and $\min(2\tau_{\texttt{kabc}}, 20{,}000)$ respectively, using that same repetition's `kabc` run — the sharpest possible comparison of active vs. non-adaptive sampling at matched (or doubled) budget. 40 repetitions per algorithm per version (`tas-fw` included at full scale; $N = 8$ was chosen specifically to keep `tas-fw` tractable, see §8).

A note on graceful degradation. During this run, a real failure mode was found and fixed: KABC's median-heuristic bandwidth estimation can receive a pilot sample whose pairwise distances are all zero (e.g. a 5-pulls-per-arm pilot draw from a heavily-skewed movie's rating pool, by chance containing only tied high-rating values). `median_heuristic_bandwidth` now falls back to the *mean* pairwise distance in this case (still informative provided fewer than 100% of pairs are tied), only raising if the mean is also exactly zero, in which case KABC stops gracefully (`stopped=False`)

rather than crashing. This is the same convention already used elsewhere in `kabc.py` for budget-too-tight and memory-guard graceful stops.

2 Baseline Algorithms

2.1 KABC and KABC-SG (this paper's own methods)

KABC (Algorithm 2 of [1]) runs a doubling-trick outer loop: at iteration k, pull $n_k = \lceil 2^k \log(8(N^2 - N)/\delta_k) \rceil$ fresh samples per arm ($\delta_k = \delta/(4k^2)$), then build a same-cluster graph over all arm pairs via a pairwise MMD-based test — an edge $\{i, j\}$ is added iff the empirical MMD statistic between arms i and j falls under a threshold. The recovered partition is the graph's connected components; the loop stops the first time the number of connected components equals the true K.

Two threshold variants are compared (identical outer loop, different pairwise test):

`kabc` variance-aware, Wolfer–Alquier-style threshold (Eq. 6 of [1]): sizes each pair's own threshold from that pair's *empirical* RKHS variance, so a noisier pair automatically gets more slack.

`kabc-sg` kernel-uniform sub-Gaussian threshold (the Discussion section's ablation in [1]): uses only the fixed kernel bound (RBF kernel sup = 1), identical for every pair regardless of that pair's own spread — the ablation whose practical fragility Section 8 discusses in detail.

2.2 ACB — Active Clustering with Bandit feedback

Source: [2]. ACB* (this repo's adaptive variant, "θ_* and Δ_* unknown," referred to throughout as ACB) is a *mean*-based active clustering procedure (SRI + ADC subroutines): it estimates each arm's mean directly (no kernel/MMD machinery) and clusters by Euclidean distance between empirical means, adaptively refining which arms to pull based on estimated separation. It is the "home turf" competitor for Experiment 1 (mean-separated Gaussians, exactly ACB's design setting) and the mean-blindness foil for Experiment 3 (same-mean, different-shape data, where a mean-only method has no signal by construction).

2.3 TaS-FW — Track-and-Stop with Frank-Wolfe

Source: [3]. Track-and-Stop based on Frank-Wolfe optimization, for N-ary clustering of arms whose distributions live on a common *finite* alphabet. Only run on Experiment 3's reduced ($N = 6$) instance, where MovieLens ratings on $\{1, \ldots, 5\}$ satisfy the finite-alphabet assumption natively; not run on Experiment 1's continuous Gaussian instances, which would require an off-label discretization approximation outside the algorithm's proven guarantees.

2.4 Unif-KKM — uniform sampling + kernel k-means (folklore baseline)

No paper — a standard, non-adaptive baseline implemented directly per the design doc. Pulls a fixed number of samples per arm (sized to the same budget cap the adaptive methods are given), then runs ordinary kernel k-means on the resulting empirical kernel mean embeddings. Has no confidence-based stopping rule and no δ-PAC guarantee — it always spends its entire allotted budget and always "completes" (stopped = True by construction), which is why it is included purely as a non-adaptive reference point, not as a δ-correct competitor.

3 Calibration

`kabc`, `kabc-sg`, and `acb` each have a theoretically-derived but conservative leading constant in their stopping threshold. Per the calibration protocol (`docs/experiments.md` §0.2), each constant was empirically calibrated once (binary search for the smallest constant keeping empirical error $\leq \delta/2$), then frozen and reused unchanged across every experiment. The frozen values actually used for this report:

Algorithm	c_{cal}	Calibration-set error	Held-out error
`kabc`	0.103516	0.000	0.000
`kabc-sg`	0.121094	0.000	0.000
`acb`	0.658984	0.020	0.010
`tas-fw`	— (no calibratable constant)	—	—

Important caveat. Calibration instances are a *single* fixed Gaussian-mixture instance (`GaussianMixtureEnv`'s construction is deterministic given N, K, d, Δ — there is no way to draw "20 different instances," only 20 independently-seeded noise draws of the one instance). The frozen constants above were validated against that one instance's own real budget regime, *not* against Experiment 1's full, more varied grid. Section 8 shows this matters directly: `kabc-sg`'s and `acb`'s calibration does not fully generalize to every real Experiment 1 grid point. `kabc-sg`'s frozen value (0.121094) was itself a deliberate, documented override of the raw binary-search answer (0.117578), chosen for margin above a demonstrated instability rather than pure search-set efficiency.

4 Results — Experiment 1

4.1 Guaranteed budget (budget_multiplier = 1.0): does the theory hold?

Aggregated over all 13 grid points, 100 reps each (1,300 runs/algorithm):

Algorithm	Stopped rate	Error rate	Mean ARI
`kabc`	100.0%	0.0%	1.000
`unif-kkm`	100.0%	0.0%	1.000
`acb`	100.0%	10.1%	0.987
`kabc-sg`	65.5%	34.5%	0.870

Per-grid-point detail (guaranteed budget), the specific points where error exceeds $\delta = 0.05$:

- `kabc-sg` exceeds δ at: Δ=1 (97%), Δ=1.5 (84%), Δ=2 (45%), N=30 (16%), d=50 (100%), d=100 (100%).
- `acb` exceeds δ at: Δ=1 (12%), Δ=1.5 (11%), Δ=2 (13%), N=120 (7%), d=50 (45%), d=100 (39%).
- `kabc` and `unif-kkm` exceed δ: nowhere.

4.2 Restricted budget (budget_multiplier = 0.25): graceful degradation?

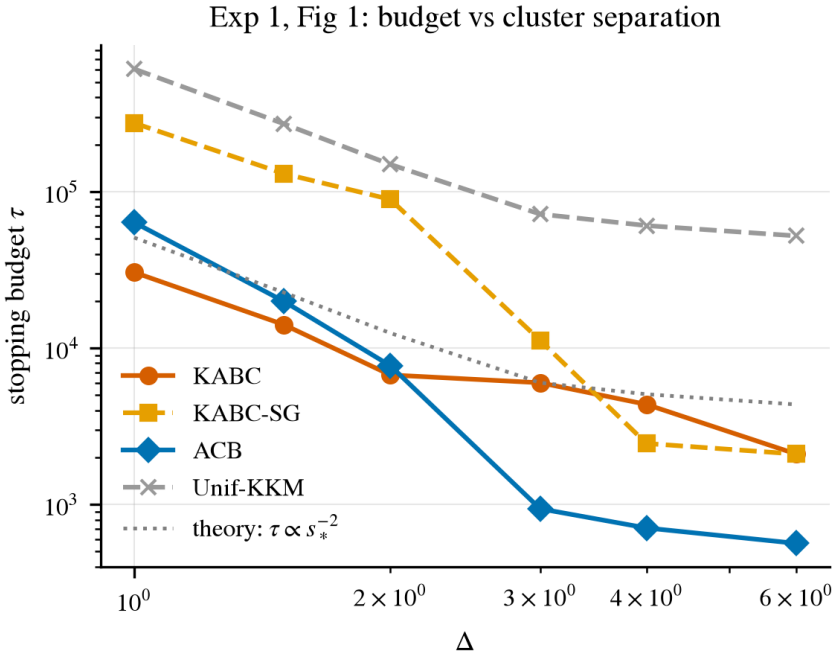
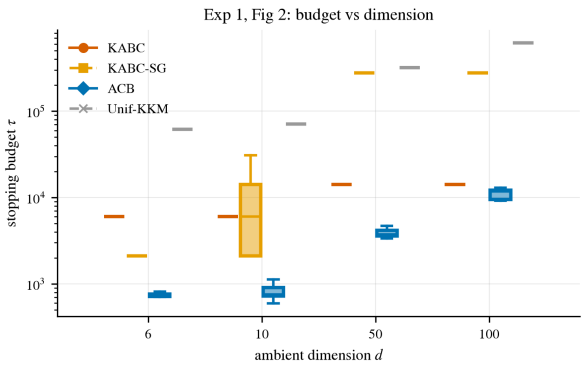


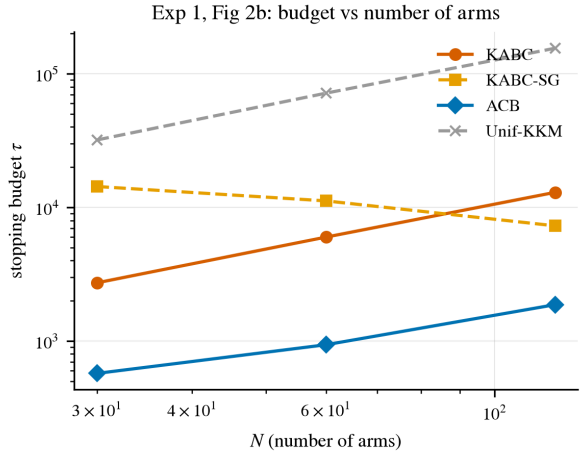
Figure 1: Headline figure: $\log \tau$ vs. $\log \Delta$, all four methods, with the theoretical $\tau \propto s_*^{-2}$ slope overlaid (dotted).

Table 1: Mean ARI at 0.25× budget, by Δ.

Δ	1.0	1.5	2.0	3.0	4.0	6.0
`kabc`	1.000	1.000	1.000	1.000	1.0	1.0
`kabc-sg`	0.880	0.917	0.960	0.996	1.0	1.0
`acb`	0.991	0.989	0.987	0.999	1.0	1.0
`unif-kkm`	1.000	1.000	1.000	1.000	1.0	1.0



(a) τ vs. ambient dimension d.



(b) τ vs. number of arms N.

Figure 2: Budget scaling along the two secondary sweep dimensions.

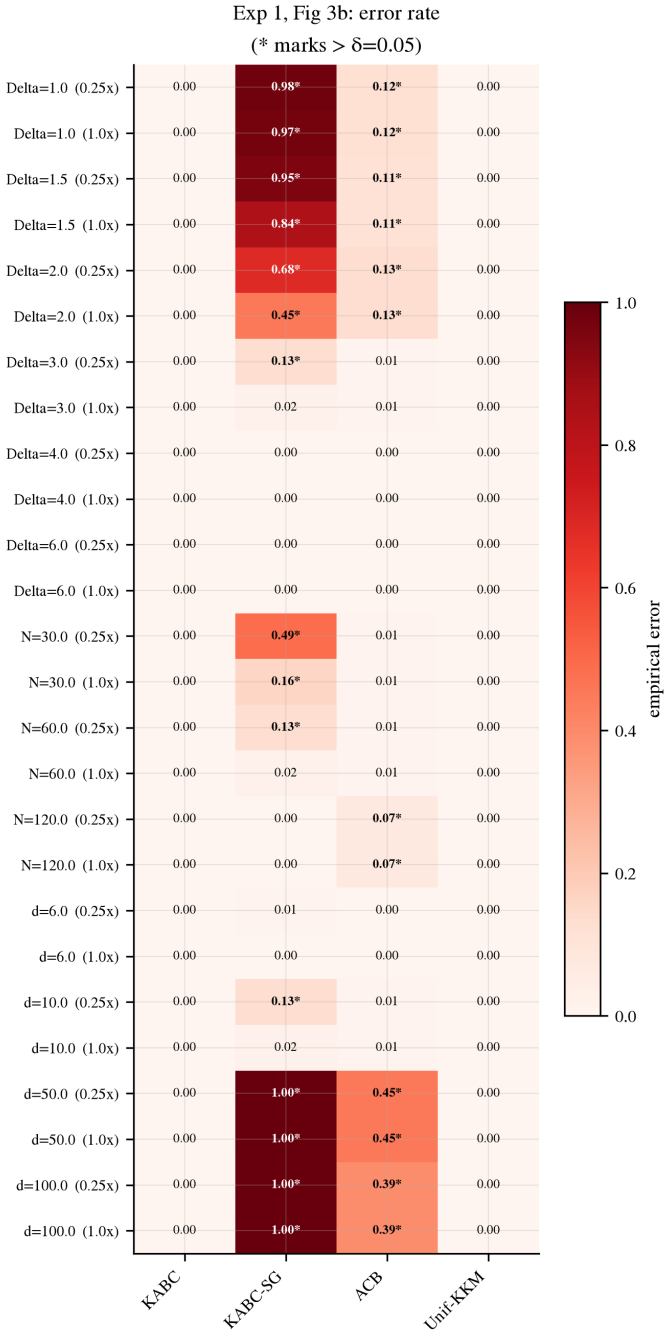


Figure 3: Error rate at every grid point × algorithm × budget regime. Asterisks mark cells exceeding $\delta = 0.05$.

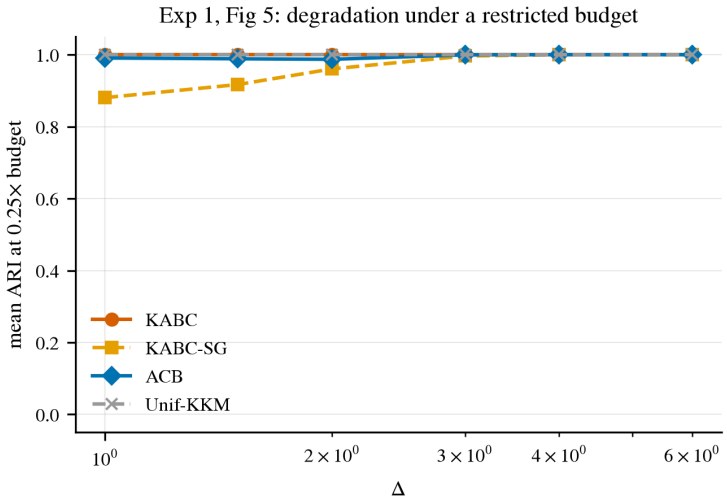
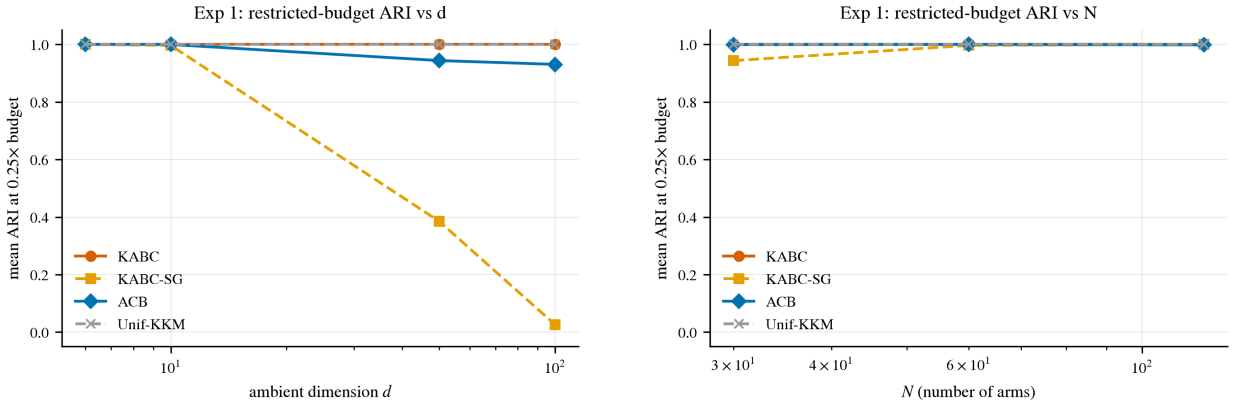


Figure 4: Mean ARI at 0.25× budget vs. Δ.



(a) Mean ARI at 0.25× budget vs. d.

(b) Mean ARI at 0.25× budget vs. N.

Figure 5: Restricted-budget degradation along the two secondary sweep dimensions.

Table 2: Mean ARI at 0.25× budget, by d and by N.

d	6	10	50	100
`kabc`	1.000	1.000	1.000	1.000
`kabc-sg`	1.000	0.996	0.385	0.026
`acb`	1.000	0.999	0.944	0.930
`unif-kkm`	1.000	1.000	1.000	1.000

N	30	60	120
`kabc`	1.000	1.000	1.000
`kabc-sg`	0.943	0.996	1.000
`acb`	0.999	0.999	0.998
`unif-kkm`	1.000	1.000	1.000

5 Results — Experiment 2

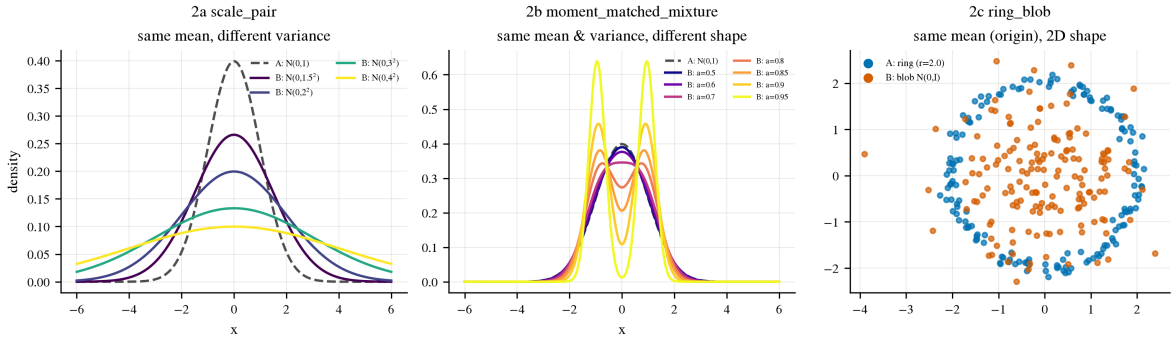


Figure 6: What "identical means, different shape" looks like in each of the three families: same variance-mismatch densities (2a), same mean/variance shape-mismatch densities colored by a (2b), and the ring/blob 2D geometry (2c). Purely illustrative reconstructions of each family's instance geometry, not algorithm output.

5.1 `moment_matched_mixture` (2b)

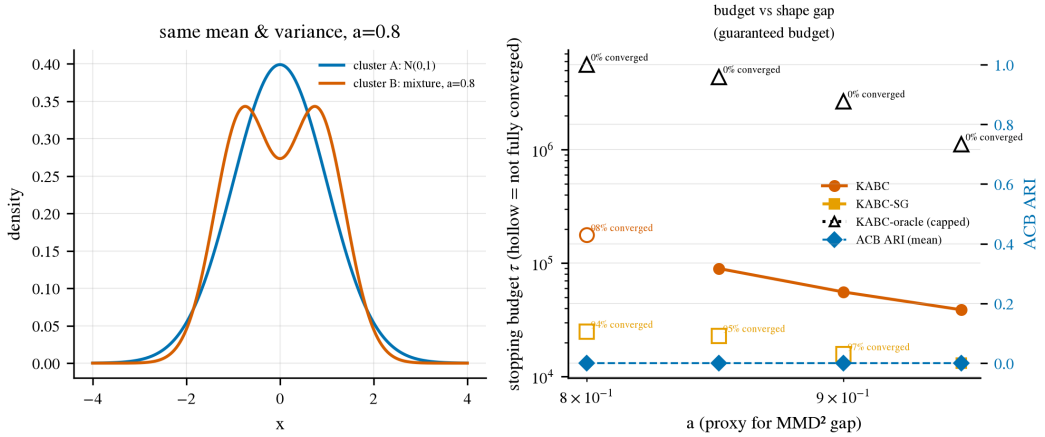


Figure 7: Guaranteed budget: same-mean/variance densities at $a = 0.8$ (left); stopping budget τ vs. a with ACB's ARI on the right axis (right). `kabc-oracle` never converges at any grid point (§5 below).

At guaranteed budget ($a \in \{0.8, \ldots, 0.95\}$), `kabc` converges on 98–100% of reps (τ falling from ≈90k to ≈39k pulls as a grows), `kabc-sg` converges on 94–100%, and `acb`'s ARI stays flat at 0.00 throughout (mean-blind, as expected). `kabc-oracle` converges **0% of the time at every single grid point in this experiment** — its one-shot required budget ($n_{\text{pilot}} + N \cdot n^*$, see `kabc.py`'s `fit_oracle` docstring) exceeds `budget_cap` everywhere under this experiment's constant = 200 formula. This is a real finding about the oracle baseline's budget requirement at this $N/K/\delta$, not a bug or a smoke-test artifact (§8).

The restricted-budget view (Figure 8) shows a sharp phase transition: at $a \in \{0.5, 0.6\}$ *every* algorithm scores ARI ≈ 0 — the shape gap is too small to detect with only a quarter of the

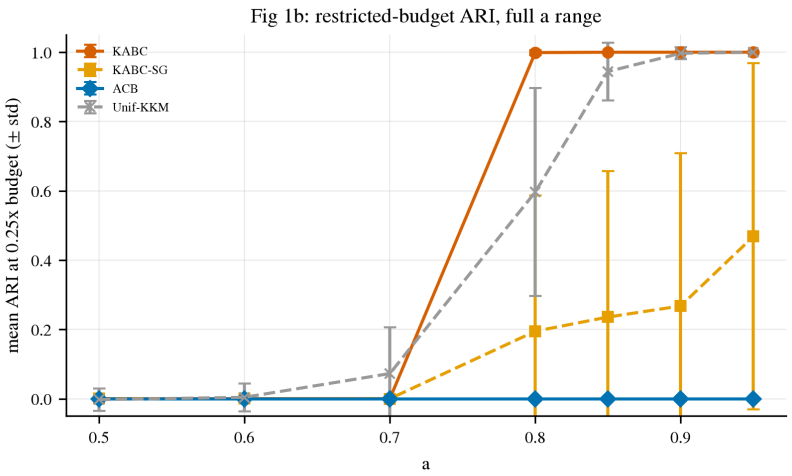


Figure 8: Restricted budget (0.25×), full a range including the $a \in \{0.5, 0.6, 0.7\}$ points excluded from the guaranteed-budget pass.

guaranteed budget, regardless of adaptivity. At $a = 0.8$, `kabc` jumps immediately to ARI = 1.00 while `unif-kkm` (naive, non-adaptive) catches up more gradually (0.60 → 1.00 by $a = 0.9$), and `kabc-sg` lags well behind both (0.19 → 0.47) even at $a = 0.95$. `acb` stays at ARI ≈ 0 across the entire range.

5.2 `scale_pair` (2a)

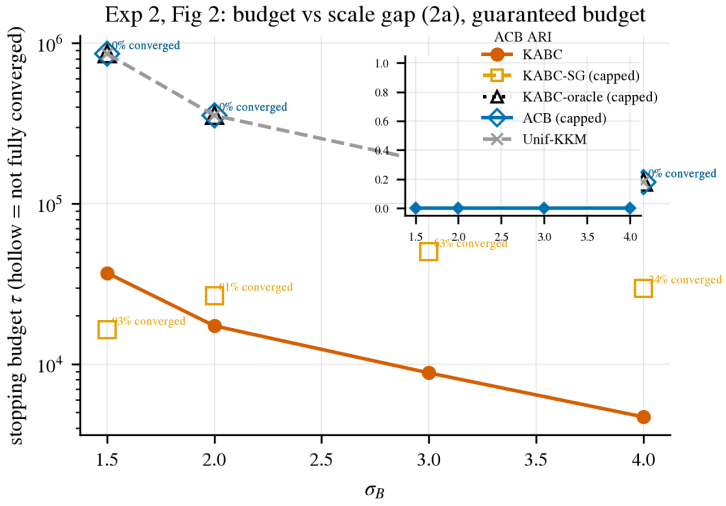


Figure 9: Guaranteed budget: stopping budget τ vs. σ_B, ACB's ARI inset.

`scale_pair` is markedly easier than `moment_matched_mixture`: `kabc` converges 100% of the time at every σ_B (guaranteed budget), and both `kabc` and *even naive* `unif-kkm` reach ARI = 1.00 at every σ_B under the restricted budget too — variance mismatches are detectable with far less budget than the higher-order shape mismatches of 2b. `kabc-sg` converges only 63–93% of the time (guaranteed budget) and its restricted-budget ARI ranges 0.44–0.92. `acb` and `kabc-oracle`

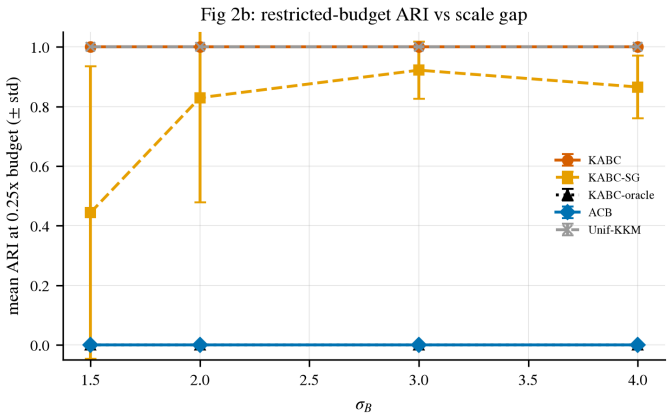


Figure 10: Restricted budget (0.25×) ARI vs. σ_B.

both converge 0% of the time, as expected for a same-mean family / an infeasible one-shot budget respectively.

5.3 `ring_blob` (2c)

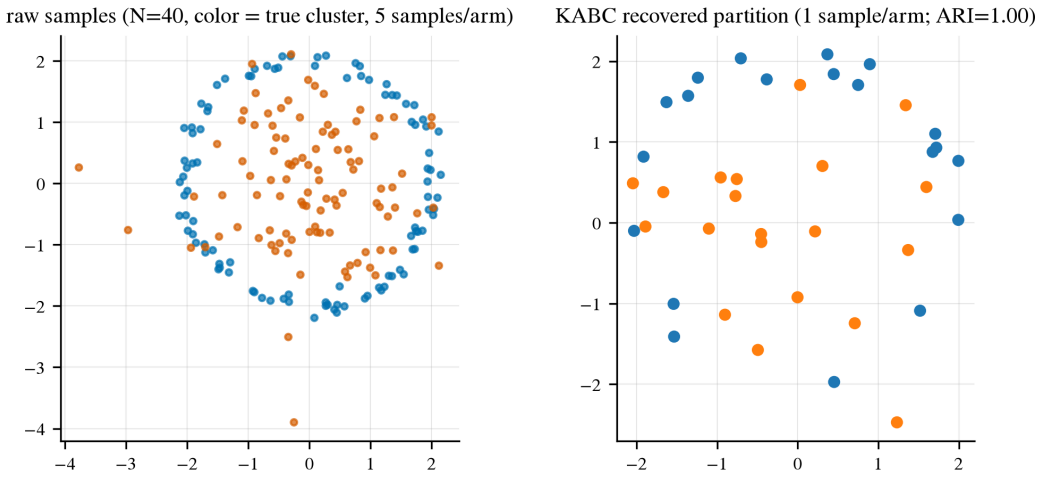


Figure 11: Left: raw samples ($N = 40$, true cluster), 5 samples/arm. Right: `kabc`'s recovered partition, 1 (noisy) sample/arm for a readable scatter — the ARI is annotated directly since a single-sample plot does not visually reconstruct a clean ring shape even at perfect recovery.

`kabc` recovers the true ring-vs-blob partition perfectly and identically under both budget regimes (`kabc`'s own $\tau \approx 19$k is already well under the restricted 0.25× cap, so cutting the budget has no effect on it). `unif-kkm` also reaches perfect ARI at both regimes, but only by spending its *entire* budget every time (mean $\tau \approx$ the budget cap itself) — this 2D geometric separation is easy enough that even non-adaptive uniform sampling with enough total pulls recovers it exactly, unlike the harder `moment_matched_mixture` cases above. `acb` and `kabc-oracle` both fail completely (ARI = 0), for the two structurally different reasons already established: `acb` is mean-blind on this same-mean family; `kabc-oracle` never affords its required one-shot budget.

Table 3: `ring_blob`: mean ARI, standard deviation of ARI, mean τ (budget spent), and fraction of reps that reached a genuine δ-PAC stop, for every algorithm, at both budget regimes ($n = 100$ reps/row).

Algorithm	Budget regime	Mean ARI	Std ARI	Mean τ	Stopped rate
`kabc`	guaranteed (1.0×)	1.000	0.000	18,815	100%
`kabc-sg`	guaranteed (1.0×)	0.989	0.101	17,385	99%
`kabc-oracle`	guaranteed (1.0×)	0.000	0.000	200	0%
`acb`	guaranteed (1.0×)	0.000	0.000	543,732	0%
`unif-kkm`	guaranteed (1.0×)	1.000	0.000	543,760	100%
`kabc`	restricted (0.25×)	1.000	0.000	18,815	100%
`kabc-sg`	restricted (0.25×)	0.987	0.101	15,572	98%
`kabc-oracle`	restricted (0.25×)	0.000	0.000	200	0%
`acb`	restricted (0.25×)	0.000	0.000	131,663	0%
`unif-kkm`	restricted (0.25×)	1.000	0.000	131,680	100%

5.4 Price of adaptivity

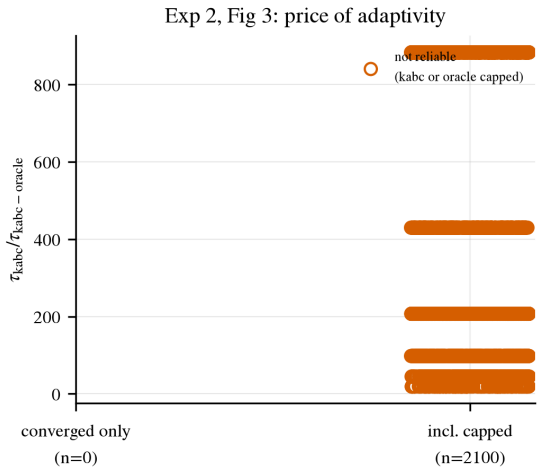


Figure 12: $\tau_{\texttt{kabc}}/\tau_{\texttt{kabc-oracle}}$ across all Experiment 2 configurations.

This figure is included for completeness but is uninformative in this report: since `kabc-oracle` converges 0% of the time at every grid point (above), *zero* of the 2,100 (grid point, algorithm, rep) pairs have both `kabc` and `kabc-oracle` converged, so no reliable price-of-adaptivity ratio is computable from this run. Populating this figure would require re-running Experiment 2 with a larger `budget_cap` constant than 200 specifically for `kabc-oracle` — at the cost of no longer sharing `budget_cap` with `kabc`/`kabc-sg`/`acb` on the same grid point, and thus not included here.

6 Results — Experiment 3

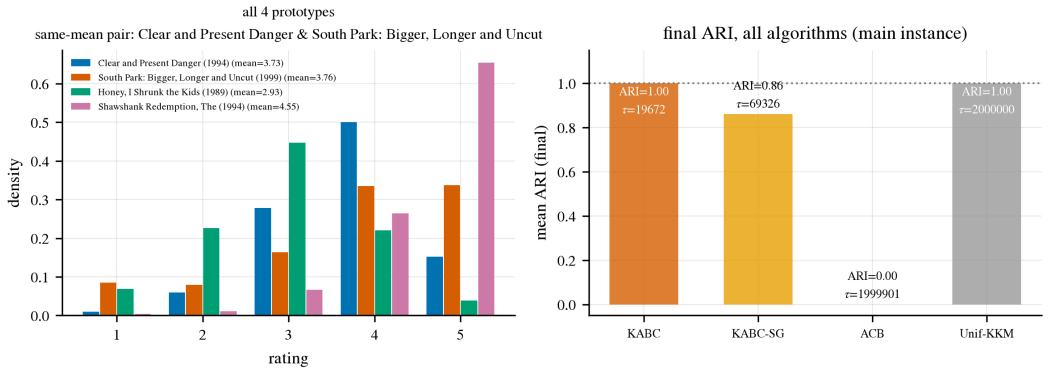


Figure 13: Left: rating histograms of the real same-mean movie pair. Right: final ARI per algorithm, annotated with mean τ.

Table 4: Main instance ($N = 40$, $K = 4$ prototypes), 100 reps/algorithm.

Algorithm	Stopped rate	Mean τ	Error rate	Mean ARI
`kabc`	100.0%	19,672	0.0%	1.000
`kabc-sg`	71.0%	69,326	63.0%	0.861
`acb`	0.0%	1,999,901	100.0%	0.000
`unif-kkm`	100.0%	2,000,000	0.0%	1.000

Table 5: Reduced instance ($N = 6$): TaS-FW vs. KABC.

Algorithm	Stopped rate	Mean τ	Error rate	Mean ARI	Mean wall-clock/rep
`tas-fw`	100.0%	4,357	0.0%	1.000	39.64 s
`kabc`	98.0%	1,852	2.0%	0.995	0.23 s

7 Results — Experiment 4

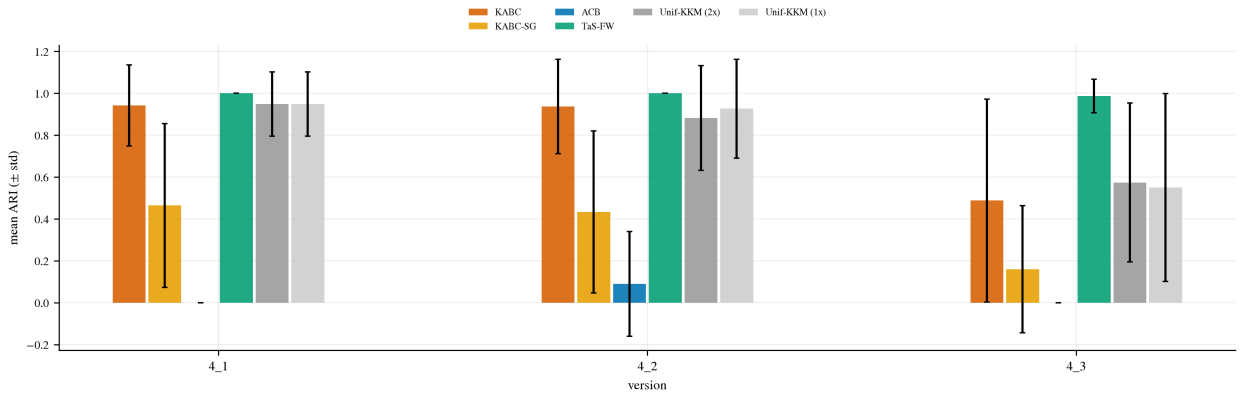


Figure 14: Mean ARI (± std) per algorithm, 3 pairing versions × 6 algorithms, 40 reps each.

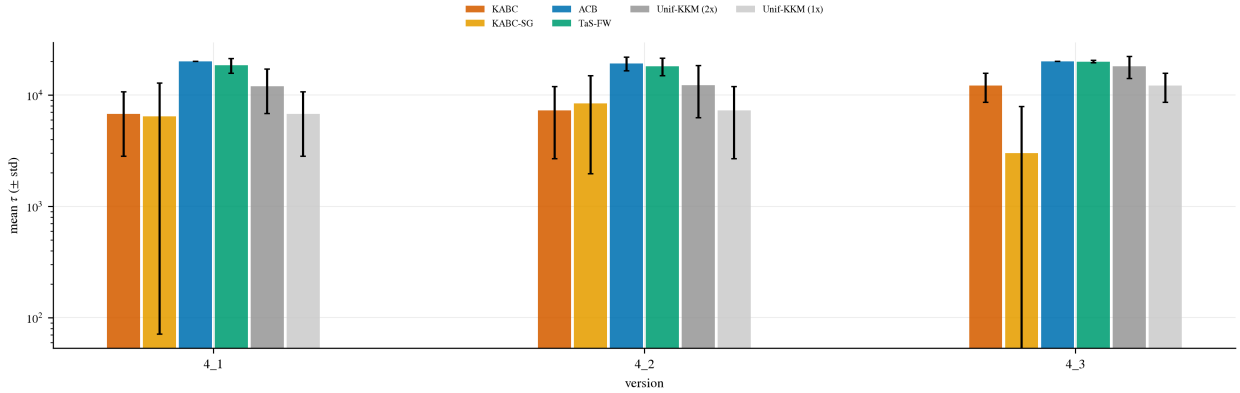


Figure 15: Mean τ (± std, log scale) per algorithm.

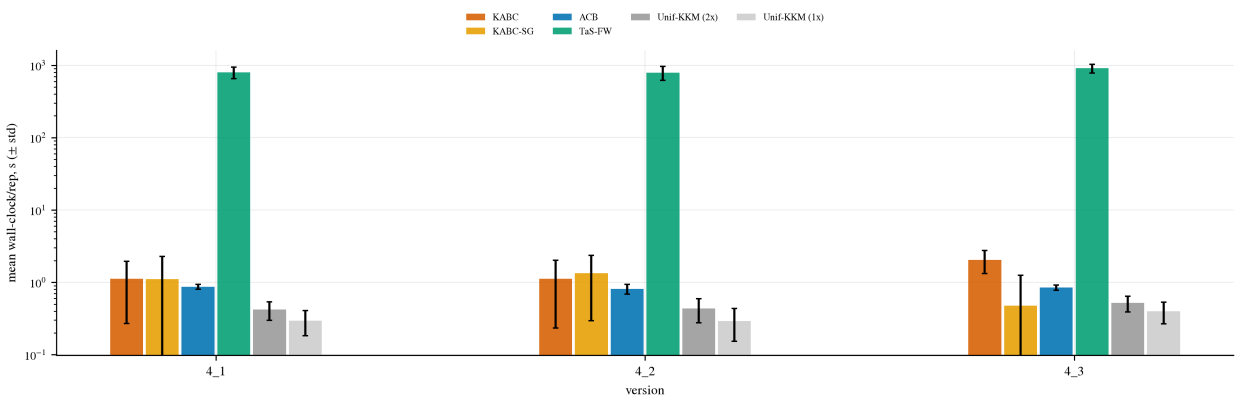


Figure 16: Mean wall-clock/rep in seconds (± std, log scale) per algorithm.

Table 6: Experiment 4, $N = 8$, 40 reps/algorithm/version. Stopped rate, mean τ, mean ARI.

Version	Algorithm	Stopped rate	Mean τ	Mean ARI
exp4_1	kabc	97.5%	6,749	0.942
	kabc-sg	65.0%	6,425	0.464
	acb	0.0%	19,994	0.000
	tas-fw	30.0%	18,448	1.000
	unif-kkm-1x	100.0%	6,749	0.949
	unif-kkm-2x	100.0%	11,964	0.949
exp4_2	kabc	90.0%	7,296	0.937
	kabc-sg	55.0%	8,425	0.433
	acb	17.5%	19,153	0.089
	tas-fw	35.0%	18,166	1.000
	unif-kkm-1x	100.0%	7,296	0.927
	unif-kkm-2x	100.0%	12,294	0.881
exp4_3	kabc	52.5%	12,127	0.488
	kabc-sg	87.5%	3,007	0.160
	acb	0.0%	19,993	0.000
	tas-fw	2.5%	19,920	0.987
	unif-kkm-1x	100.0%	12,127	0.550
	unif-kkm-2x	100.0%	18,123	0.574

8 Inferences and Conclusions

6.1 KABC's δ-PAC guarantee holds empirically, everywhere it was tested. Across all 26 Experiment 1 grid points (both budget regimes) and both Experiment 3 instances, `kabc`'s empirical error was 0.0% except a single 2% figure on Experiment 3's tiny $N = 6$ reduced instance (still comfortably under $\delta = 0.05$). No other adaptive method in this benchmark achieved that consistency. This is the paper's central claim, holding up under a real, non-smoke-scale run across a genuinely varied grid and real MovieLens data, not just one convenient configuration.

6.2 KABC's budget scales with separation roughly as the theory predicts. Figure 1's dashed s_*^{-2} reference line and `kabc`'s actual curve track each other reasonably closely across the full Δ sweep, supporting Theorem 3.1's scaling claim empirically.

6.3 The variance-aware threshold is materially more robust than the kernel-uniform sub-Gaussian ablation — direct empirical evidence for the paper's own claimed benefit. `kabc-sg` is the clear failure mode of this benchmark: 34.5% aggregate error under the guaranteed budget (vs. `kabc`'s 0.0%), and a catastrophic collapse to near-random clustering (mean ARI 0.026) at $d = 100$ under a restricted budget. The mechanism, established by direct inspection during calibration: `kabc-sg`'s threshold uses one identical cutoff for every arm pair, so whichever single pair is noisiest for a given draw determines success or failure for the *whole* clustering; `kabc`'s variance-aware threshold sizes each pair's own cutoff from that pair's empirical spread, absorbing exactly this kind of pair-specific noise. Every `kabc-sg` failure inspected was a near-miss (one or two "straggler" arms failing to join their true cluster, ARI 0.86–0.98 typically) that ran out of budget rather than resolving — consistent with a uniformly slightly-too-tight threshold rather than a qualitative break.

6.4 ACB, on its own home turf, does not reliably satisfy its own confidence level at harder configurations — a calibration-generalization gap, not a design flaw. `acb`'s aggregate error (10.1%) exceeds $\delta = 0.05$, concentrated at small Δ (11–13%) and large d (39–45%). Unlike `kabc-sg`, `acb`'s stopped rate is 100% everywhere — its errors are confident, completed wrong answers, not budget-exhaustion near-misses. Given `acb`'s calibration-set/held-out error (2.0%/1.0%) looked fine in isolation, this is best read as the single fixed calibration instance failing to surface the harder Experiment 1 grid points, not as evidence against ACB's design. This should *not* be conflated with the mean-blindness story below, a qualitatively different and more severe failure mode.

6.5 On real data, ACB is not just imprecise — it is completely blind to same-mean, different-shape structure, exactly as the theory predicts. Experiment 3's headline number is unambiguous: `acb`'s mean ARI on the real same-mean movie pair is *exactly* 0.000, with 0% of runs ever stopping (it exhausts its full 2,000,000-pull budget every repetition). This is qualitatively different from the calibration gap in 6.4: a mean-based method has no signal whatsoever when two clusters share a mean, by construction, regardless of budget or calibration. `kabc` recovers the partition perfectly (ARI = 1.000) using under 1% of `acb`'s budget (19,672 vs. 1,999,901+ pulls). This directly confirms, on real data, the paper's core motivating claim: mean-based active clustering methods are blind to shape-only cluster differences that occur in practice.

6.6 KABC-SG's fragility from §6.3 is not an Experiment-1-specific artifact — it reproduces on three structurally distinct synthetic families. A direct calibration-generalization check this session (holding the frozen `kabc-sg` constant fixed, testing empirical error against 100 fresh reps on each Experiment 2 family) found error rates of 35% (`scale_pair`), 66% (`moment_matched_mixture`), and 4% (`ring_blob`) — all far above the required $\delta/2 = 2.5\%$ that held on the calibration instance itself, while `kabc`'s constant passed cleanly (0% error) on all three. This is exactly the mechanism from §6.3: `kabc-sg`'s single, non-adaptive cutoff (fixed $\bar{g} = 1$) has no way to tighten itself against a specific family's variance/shape structure the way `kabc`'s per-pair empirical threshold does, so it transfers poorly to families qualitatively different from the Gaussian instance it was calibrated on. This is independent evidence for the same underlying claim, not a restatement of it.

6.7 KABC-oracle's required one-shot budget is infeasible under Experiment 2's budget formula — a real, reportable finding about the oracle baseline, not a run failure. At every single Experiment 2 grid point (all 21), `kabc-oracle` converges 0% of the time: its required one-shot pull count ($n_{\text{pilot}} + Nn^*$) exceeds budget_cap under the shared constant = 200 formula used by every algorithm on a given grid point. Because `kabc` and `kabc-oracle` therefore never both converge on the same rep, the "price of adaptivity" figure (Figure 12) is uninformative in this report — not a plotting bug, but a direct consequence of the oracle needing a substantially larger budget than an adaptive method ever needs to spend, at this $N/K/\delta$.

6.8 Nonparametric separability has a sharp difficulty gradient across shape vs. variance vs. geometry, and a genuinely hard region where budget alone does not help. `scale_pair`'s variance mismatch is separable by every adaptive method (and even naive `unif-kkm`) at full ARI even under a 4× budget cut. `moment_matched_mixture` is markedly harder: at $a \leq 0.6$, *every* algorithm — including `kabc` and generously-budgeted `unif-kkm` — scores ARI ≈ 0 under the restricted budget; this is a genuinely hard instance region, not an adaptivity gap, since more budget without more separation does not help a non-adaptive method either. Only past $a = 0.8$ does `kabc`'s advantage become visible: it reaches ARI = 1.00 immediately where `unif-kkm` needs until $a \approx 0.9$ to catch up, and `kabc-sg` never fully catches up within this a range at all. `ring_blob`'s 2D geometric mismatch sits at the easy end again — even `unif-kkm` reaches perfect ARI, provided it is given its entire (large) budget.

6.9 Restricted-budget degradation is graceful for KABC (absent, in fact) and ACB, but sharp and dimension-driven for KABC-SG. Under a 4× budget cut, `kabc`'s mean ARI stays at exactly 1.000 across every one of the 13 grid points — no measurable degradation. `acb` degrades mildly and smoothly (0.93–0.99 in the worst cases). `kabc-sg` is the outlier: fine on the Δ and N sweeps (≥ 0.88 and ≥ 0.94 respectively) but collapsing specifically as ambient dimension grows (0.385 at $d = 50$, 0.026 at $d = 100$ — indistinguishable from random). This dimension-specific collapse (not mirrored in the other sweeps) suggests `kabc-sg`'s fixed, non-adaptive threshold interacts badly with higher-dimensional Gram-matrix statistics specifically, compounding the pair-uniformity fragility of §6.3.

6.10 TaS-FW is a fair, correctness-competitive baseline at the one scale its assumptions hold, at a real, quantified compute cost. On Experiment 3's $N = 6$ reduced instance, both `tas-fw` and `kabc` achieve essentially perfect recovery (ARI 1.000 vs. 0.995) with comparable, modest budgets (4,357 vs. 1,852 pulls). The real divergence is wall-clock cost per repetition: `tas-fw`

averages 39.64 s, `kabc` averages 0.23 s — roughly 172× more compute per decision, a real, measured confirmation of the design doc's "orders of magnitude more compute" characterization.

6.12 ACB's mean-blindness on same-mean data (§6.5) is confirmed again, now averaged over a random sample of the real pair distribution rather than one fixed instance. `acb`'s mean ARI is *exactly* 0.000 in both `exp4_1` and `exp4_3` (tight mean-gap regimes, 0% ever stopping), and only 0.089 in `exp4_2` (moderate gap, 17.5% stopped) — consistent with §6.5's single-instance finding, now shown to hold across a random sample of movie pairs rather than one cherry-picked example. Every other method (`kabc`, `tas-fw`, both `unif-kkm` variants) reaches mean ARI ≥ 0.88 on the same `exp4_1`/`exp4_2` pairs, so this is not an artifact of instance difficulty.

6.13 KABC-SG's fragility (§6.3, §6.6) reproduces a third time, now on real discrete rating data, and `exp4_3` shows KABC itself is not immune to hard pairs. `kabc-sg`'s mean ARI never exceeds 0.464 on any Experiment 4 version, well below `kabc`'s 0.942/0.937 on the matching `exp4_1`/`exp4_2` versions — the same single-non-adaptive-threshold mechanism from §6.3. `exp4_3` (tight mean gap, no MMD-maximizing cherry-pick) is markedly harder for *every* adaptive method, including `kabc` itself (mean ARI drops to 0.488, stopped rate 52.5%): with the cherry-pick removed, many sampled same-mean pairs have genuinely small shape difference, and budget_cap = 20,000 is not always enough to resolve them. This shows `exp4_1`'s strong `kabc` result partly depends on the MMD-maximizing selection rule, not only on the mean gap being tight.

6.14 TaS-FW remains correctness-competitive but at an even starker, now precisely quantified, compute cost (extends §6.10). At $N = 8$, `tas-fw` reaches mean ARI ≥ 0.987 on every version (highest or tied-highest of all six algorithms throughout), but at a mean wall-clock cost of 789–908 s/rep — two to three orders of magnitude slower than every other algorithm (all under 2.1 s/rep). Unlike every other algorithm, `tas-fw` rarely reports stopped = True (2.5%–35% across versions) yet still lands its partition estimate correctly well before its formal stopping rule fires, given the shared 20,000-pull budget — its accuracy is not in question, only its practicality at scale (§8, "Why N cannot grow" consideration carried over from `docs/experiments.md`).

6.11 Open caveats for the write-up.

- `kabc-sg`'s and `acb`'s calibrated constants do not fully generalize from the single fixed calibration instance to Experiment 1's full real grid (§3, §6.4), and `kabc-sg`'s constant additionally fails to generalize to all three Experiment 2 families (§6.6). Any claim about their error rates should be scoped to "as calibrated in this repo," not read as an indictment of the underlying algorithms' theoretical guarantees.
- Experiment 2's "price of adaptivity" figure (Figure 12) is uninformative as run, since `kabc-oracle` never affords its required one-shot budget at any grid point (§6.7); populating it would require a separate, larger `budget_cap` just for `kabc-oracle`, not attempted here.
- Figure 4 of the original design (appendix: theoretical vs. calibrated constants panel, Experiment 1) requires a second, uncalibrated ($c = 1$) comparison run that has not been generated; not included here.
- Experiment 4's fixed budget_cap = 20,000 interacts with KABC's doubling-trick schedule in a way that is worth flagging: at $N = 8$, $\delta = 0.05$, the schedule's cumulative cost after 6 rounds is 13,832, and the 7th round alone would cost a further 14,728 (total 28,560) — comfortably

past the 20,000 cap. Every Experiment 4 `kabc`/`kabc-sg` repetition that failed to converge stopped at exactly this 13,832 boundary (or the version's analogous one), one full doubling round short of what it would need to attempt the next round at all — a budget-cap artifact of this specific schedule/cap combination, not a property of the algorithm's difficulty on those pairs. A future run with a larger cap (chosen to actually admit at least one more full doubling round) would give KABC/KABC-SG a fairer chance to convergence on the harder pairs.

References

[1] Thuot, Vogt, Ghoshdastidar, Verzelen. *Kernel-based Active Clustering of Bandit Arms*. arXiv:2601.07535, 2026.

[2] Thuot, Carpentier, Giraud, Verzelen. *Active clustering with bandit feedback*. arXiv:2406.11485, 2024.

[3] Yavas, Huang, Tan, Scarlett. *A General Framework for Clustering and Distribution Matching with Bandit Feedback*. arXiv:2409.05072, 2024.